```
Output:
I've always found that writing comes from a great inner pain.
I'm a writer, you monsters! I create!
```

Cast your mind back to the section "Local Scope and Static Variables" in Chapter 3, "Functions," where I demonstrated a trick for getting locally scoped variables to persist across multiple calls to a function. We saw a function `getCount()` that initialized a counter variable like this: `static $counter = 0`, which meant that `$counter` retained its value even though it went out of scope (so it could be used to store data between calls). Static properties behave in a similar way. Here's an OOP version of the `getCount()` function:

```
class Counter
{
    private static int $count = 0;

    public function getCount(): int
    {
        return $this::$count++;
    }
}

$counter1 = new Counter();
$counter2 = new Counter();

print "counter1 has count: " . $counter1->getCount() . PHP_EOL;
print "counter2 has count: " . $counter2->getCount() . PHP_EOL;
print "counter1 has count: " . $counter1->getCount() . PHP_EOL;

Output:
counter1 has count: 0
counter2 has count: 1
counter1 has count: 2
```

We're calling a `getCount()` on different object instances of the same class, but because the internal `$count` property is `static` it exists in class context so all instances share the same data. Notice the use of `$this::` in the implementation of `getCount()`, the scope resolution operator is required for accessing static class members during execution of object context methods. There is, however, a slight hitch if you need to access class members from a method that is also static…